

Testing Graph Databases with Synthesized Queries

Zijing Yin, Si Liu, David Basin

ETH Zurich

SIGMOD 2025

Graph Databases and Cypher Query Language

Graph Databases (GDBs) store data as:

- **Nodes** (e.g., USER, MOVIE)
- **Relations** (e.g., LIKES, FRIEND)
- **Properties** on both (e.g., name='Alice Margatroid', rating=10)



Graph Databases and Cypher Query Language

Graph Databases (GDBs) store data as:

- **Nodes** (e.g., USER, MOVIE)
- **Relations** (e.g., LIKES, FRIEND)
- **Properties** on both (e.g., name='Alice Margatroid', rating=10)



Query Language: Cypher

- Clause-based: MATCH, WHERE, WITH, UNWIND, RETURN
- Example:

```
MATCH (u:USER)-[r:LIKES]->(m:MOVIE)
WHERE u.name = 'Alice Margatroid' AND r.rating >= 8
UNWIND m.genres AS g
RETURN DISTINCT m.name, g
```

Challenges in Graph Database Testing

GDBs' advanced query features make them prone to **logic bugs**:

- Database returns *incorrect results silently* (no crash, no error).
- Hard to detect without a reliable oracle.

Challenges in Graph Database Testing

GDBs' advanced query features make them prone to **logic bugs**:

- Database returns *incorrect results silently* (no crash, no error).
- Hard to detect without a reliable oracle.

Differential Testing (e.g., GDsmith)

- Run same query on multiple GDBs → compare results.
- **False negatives**: Shared bugs missed.
- **False positives**: Semantic differences treated as bugs.

Challenges in Graph Database Testing

GDBs' advanced query features make them prone to **logic bugs**:

- Database returns *incorrect results silently* (no crash, no error).
- Hard to detect without a reliable oracle.

Differential Testing (e.g., GDsmith)

- Run same query on multiple GDBs → compare results.
- **False negatives**: Shared bugs missed.
- **False positives**: Semantic differences treated as bugs.

Metamorphic Testing (e.g., GDBMeter, GRev)

- Rewrite query (e.g., partition, reorder) → check semantic relations.
- **Still no absolute correctness**: Both original and rewritten queries can be wrong.
- **Limited query complexity** due to rewrite constraints.

Challenges in Graph Database Testing

GDBs' advanced query features make them prone to **logic bugs**:

- Database returns *incorrect results silently* (no crash, no error).
- Hard to detect without a reliable oracle.

Differential Testing (e.g., GDsmith)

- Run same query on multiple GDBs → compare results.
- **False negatives**: Shared bugs missed.
- **False positives**: Semantic differences treated as bugs.

Metamorphic Testing (e.g., GDBMeter, GRev)

- Rewrite query (e.g., partition, reorder) → check semantic relations.
- **Still no absolute correctness**: Both original and rewritten queries can be wrong.
- **Limited query complexity** due to rewrite constraints.

→ Need a testing method with *established ground truth*.

Challenges and Author's Solution

Core Challenge: How to establish a reliable oracle that defines the *correct* result of a complex graph query?

Challenges and Author's Solution

Core Challenge: How to establish a reliable oracle that defines the *correct* result of a complex graph query?

Author's Solution: GQS (Graph Query Synthesis)

Synthesize queries from a known ground truth.

- Start with a randomly generated graph.
- **Establish ground truth:** Select a set of expected property-value pairs.
- **Synthesize a complex query** that *should* return this exact ground truth.
- Any discrepancy between actual result and ground truth = **logic bug**.

Challenges and Author's Solution

Core Challenge: How to establish a reliable oracle that defines the *correct* result of a complex graph query?

Author's Solution: GQS (Graph Query Synthesis)

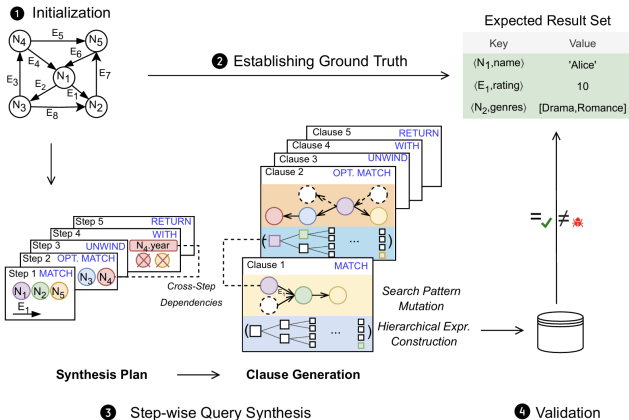
Synthesize queries from a known ground truth.

- Start with a randomly generated graph.
- **Establish ground truth:** Select a set of expected property-value pairs.
- **Synthesize a complex query** that *should* return this exact ground truth.
- Any discrepancy between actual result and ground truth = **logic bug**.

Why this approach works: GQS establishes a **definitive correctness oracle** by **simultaneously controlling** (1) the **input graph data** and (2) the **expected query result (ground truth)**.

- Any mismatch between the actual output and the ground truth **unambiguously indicates a logic bug**.
- Unlike differential or metamorphic methods, it does **not rely on assumptions** about other systems or query transformations.

GQS Workflow: An Overview



- 1 **Initialization:** Generate a random graph and load it into the GDB.
- 2 **Establishing Ground Truth:** Select properties to form the expected result set.
- 3 **Step-wise Query Synthesis:** Build a complex query that should return the ground truth.
- 4 **Validation:** Execute the query and compare the result with the ground truth.

Stage 1: Graph Generation and Abstraction

Goal: Create a small, random labeled property graph (LPG).

- Randomly generate nodes, relations, labels, and properties.
- Load the graph into the GDB under test.

Stage 1: Graph Generation and Abstraction

Goal: Create a small, random labeled property graph (LPG).

- Randomly generate nodes, relations, labels, and properties.
- Load the graph into the GDB under test.

Abstraction via Operations: All query synthesis is managed through abstract operations.

Operation	Clause	Example
$\mathcal{E}^+$	(OPTIONAL) MATCH	MATCH (n1:Person)
$\mathcal{E}^-$	WITH, RETURN	RETURN n2 (n1 not returned)
$\mathcal{A}^+$	WITH, RETURN	WITH n.name AS Name
$\mathcal{A}^-$	WITH, RETURN	RETURN n (Name not returned)
$\mathcal{L}^+$	UNWIND	UNWIND [1,2,3] AS num
$\mathcal{L}^-$	WHERE	WHERE num <= 2 (truncate list)

This abstraction ensures that any element introduced ($\mathcal{E}^+$, $\mathcal{A}^+$, $\mathcal{L}^+$) can be later removed ($\mathcal{E}^-$, $\mathcal{A}^-$, $\mathcal{L}^-$) to guarantee the final result matches the ground truth.

Stage 2: Building Valid Cross-Step Dependencies

Challenge: The order of operations matters!
(e.g., you cannot access a property before its node is introduced).

Solution: Model dependencies with a DAG and schedule operations using topological sorting.

- **Hard Constraint ($\prec$):** An operation *must* come before another.
 - $N^+ \prec (N.name)^+$
- **Weak Constraint ($\preceq$):** An operation can be in the same or a later step.
 - $(N.name)^+ \preceq N^-$

Stage 2: Building Valid Cross-Step Dependencies

Challenge: The order of operations matters!
(e.g., you cannot access a property before its node is introduced).

Solution: Model dependencies with a DAG and schedule operations using topological sorting.

- **Hard Constraint ($\prec$):** An operation *must* come before another.
 - $N+ \prec (N.name)+$
- **Weak Constraint ($\preceq$):** An operation can be in the same or a later step.
 - $(N.name)+ \preceq N-$

Algorithm 1: Operation Scheduling

- Input: DAG of operations and constraints.
- Output: An array Step (operations per clause) and Var (available variables per step).
- Process: Repeatedly pick ready-to-schedule operations (in-degree 0), respecting clause types and constraints.

Stage2: Algorithm 1: Scheduling operations

- Include all *essential operations* to access the ground truth (e.g., N^+ , $(N.name)^+$).
- Randomly inject *supplementary operations* (e.g., A^+ , L^+) to increase complexity.

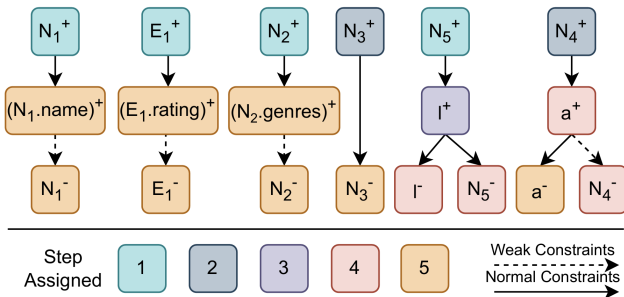
Algorithm 1 Scheduling operations

Input: G , a DAG comprising operations as nodes and their constraints as edges

Output: $Step$, an array with the operations assigned to each step; Var , an array with the referenceable variables at each step

```
1:  $i \leftarrow 1$  ▷ the current step
2: while  $G$  is not empty do
3:    $Step[i] \leftarrow \emptyset$  ▷ initializing current step's ops
4:   for  $o$  in  $G$  do
5:     if  $deg^-(o) = 0 \wedge align(Step[i], o) \wedge rand\_bool()$  then
6:        $Step[i].append(o)$ 
7:       for  $o'$  in  $o.weak\_related$  do ▷  $o \preceq o'$ 
8:         if  $deg^-(o') = 1 \wedge align(Step[i], o') \wedge rand\_bool()$  then
9:            $Step[i].append(o')$ 
10:        end if
11:      end for
12:    end if
13:  end for
14:   $Var[i] \leftarrow ref\_vars(Var[i-1], Step[i])$  ▷  $Var[0] = \emptyset$ 
15:   $G.remove(Step[i])$ 
16:   $i \leftarrow i + 1$ 
17: end while
```

Stage 2: Example of Scheduling



Result from Algorithm 1:

- Step1: $\{N1+, E1+, N2+, N5+\} \rightarrow \text{Var1} = \{N1, E1, N2, N5\}$
- Step2: $\{N3+, N4+\} \rightarrow \text{Var2} = \{N1, E1, N2, N5, N3, N4\}$
- Step3: $\{I+\} \rightarrow \text{Var2} = \{N1, E1, N2, N5, N3, N4, I\}$
- Step4: $\{I-, N5-, a+, N4-\} \rightarrow \text{Var4} = \{N1, E1, N2, N3, a\}$
- Step5: $\{N1.name+, \dots\} \rightarrow \text{Var5} = \{N1.name, E1.rating, N2.genres\}$

The final Var contains only the ground truth properties, ready for the RETURN clause.

Stage 3: Synthesizing Complex Search Patterns

Goal: Make the MATCH clause complex to trigger deep GDB bugs.

Step 1: Find Base Patterns

- For a set of nodes to be introduced (e.g., N3, N4), find simple paths connecting them to nodes in the current Var.
- Use BFS/DFS on the small graph. Paths are *base patterns*.

Stage 3: Synthesizing Complex Search Patterns

Goal: Make the MATCH clause complex to trigger deep GDB bugs.

Step 1: Find Base Patterns

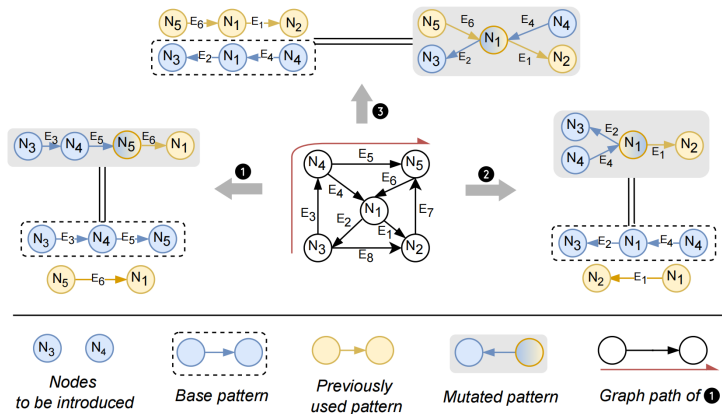
- For a set of nodes to be introduced (e.g., N3, N4), find simple paths connecting them to nodes in the current Var.
- Use BFS/DFS on the small graph. Paths are *base patterns*.

Step 2: Mutation with Previous Patterns

- Combine the new *base pattern* with an existing *previous pattern* from the query.
- Three mutation strategies based on the shared node's position:
 - 1 **Concatenation:** Shared node is an endpoint of both paths.
 - 2 **Branching:** Shared node is an endpoint of one path only.
 - 3 **Crossing:** Shared node is in the middle of both paths.

This creates highly interconnected and non-trivial graph traversal patterns.

Stage 3: Pattern Mutation Strategies



1 Concatenation

2 Branching

3 Crossing

These mutations ensure the final query is complex and exercises a wide variety of the GDB's query planning and execution logic.

Stage 3: From Scheduling to Synthesized Cypher Query

Based on the example, GQS constructs a concrete query step-by-step:

Step 1: Introduce N1, E1, N2, N5

```
MATCH (n1)-[e1]->(n2), (n5)-[e2]->(n1)
```

```
WHERE n1.id = 1 AND e1.id = 1 AND n2.id = 2 AND n5.id = 5
```

Stage 3: From Scheduling to Synthesized Cypher Query

Based on the example, GQS constructs a concrete query step-by-step:

Step 1: Introduce N1, E1, N2, N5

```
MATCH (n1)-[e1]->(n2), (n5)-[e2]->(n1)
WHERE n1.id = 1 AND e1.id = 1 AND n2.id = 2 AND n5.id = 5
```

Step 2: Introduce N3, N4

```
MATCH (n3)-[e3]->(n4), (n4)-[e4]->(n5), (n5)-[e2]->(n1)
WHERE n3.id = 3 AND n4.id = 4
```

Stage 3: From Scheduling to Synthesized Cypher Query

Based on the example, GQS constructs a concrete query step-by-step:

Step 1: Introduce N1, E1, N2, N5

```
MATCH (n1)-[e1]->(n2), (n5)-[e2]->(n1)
WHERE n1.id = 1 AND e1.id = 1 AND n2.id = 2 AND n5.id = 5
```

Step 2: Introduce N3, N4

```
MATCH (n3)-[e3]->(n4), (n4)-[e4]->(n5), (n5)-[e2]->(n1)
WHERE n3.id = 3 AND n4.id = 4
```

Step 3–4: Expand list & project variables

```
UNWIND [1,2,3] AS i
WITH n1, e1, n2, n3, n4.year AS a
```

Stage 3: From Scheduling to Synthesized Cypher Query

Based on the example, GQS constructs a concrete query step-by-step:

Step 1: Introduce N1, E1, N2, N5

```
MATCH (n1)-[e1]->(n2), (n5)-[e2]->(n1)
WHERE n1.id = 1 AND e1.id = 1 AND n2.id = 2 AND n5.id = 5
```

Step 2: Introduce N3, N4

```
MATCH (n3)-[e3]->(n4), (n4)-[e4]->(n5), (n5)-[e2]->(n1)
WHERE n3.id = 3 AND n4.id = 4
```

Step 3–4: Expand list & project variables

```
UNWIND [1,2,3] AS i
WITH n1, e1, n2, n3, n4.year AS a
```

Step 5: Return ground truth

```
RETURN n1.name AS name, e1.rating AS rating, n2.genres AS genres
```


Stage 4: Generating Complex WHERE Clauses

Goal: Stress-test GDBs' expression evaluators by replacing simple filters with deeply nested equivalents.

Approach (simplified in practice):

- Start from a base expression (e.g., `n4.id = 4`).
- Iteratively wrap it with randomly selected **mathematical/string functions**:
 - Examples: `sqrt()`, `char_length()`, `left( , )`, `sign()`, etc.
- After each wrapping, ensure the expression remains *semantically equivalent* for filtering.
- Repeat up to a target depth (e.g., 5–8 levels).

```
WHERE sqrt(char_length(left(m.name, n4.id))) = 2.0
```

This generates highly complex predicates that effectively expose expression-evaluation bugs.

Experimental Results: Bug Discovery

GQS found 36 previously unknown bugs in production GDBs!

GDB	Logic Bugs	Other Bugs	Total
Neo4j	2	3	5
Memgraph	6	1	7
Kùzu	5	2	7
FalkorDB	13	4	17
Total	26	10	36

Table: Summary of bugs detected by GQS (Table 3 from paper).

- **Logic bugs (26):** Incorrect query results.
- **Other bugs (10):** Crashes, memory leaks, exceptions.
- **Long-latent bugs:** Some existed for up to 5 years.

Example Bug 1: FalkorDB

```
1 MATCH (n2)<-[r1]-(n0), (n3)-[r2]->(n4)-[r3]->(n5)
   WHERE r1.id=13
2 UNWIND [n5.k2 <> r3.id, false] as a1
3 WITH DISTINCT n2, r3, n3, n4, n5, endNode(r1) as a2, n0
4 MATCH (n2)<-[r4 :T10]-(n0), (n3)-[r5]->(n4)-[r6]->(n5)
   WHERE (((r6.k85)+(n2.k11)) ENDS WITH 'qIIcZH6h') AND
   ((n2.k9) = -1982025281) AND (n5.k2<=-881779936)
5 RETURN n2.id as a3, r6.id as a4
6 // expected result: {a3:1, a4:16} ✓
7 // actual result: {a3:4, a4:16} ✗
```

Figure: A 4-year-old logic bug in FalkorDB (Fig. 1 from paper).

The Synthesized Query:

- Contains multiple MATCH clauses, UNWIND, WITH.
- Uses complex nested expressions and cross-clause variable references.

The Bug:

- FalkorDB returns an incorrect value for field a3.
- Expected: 1, Actual: 4.

Example Bug 2: Memgraph

```
1 MATCH (n0 :L0 :L6 :L11)<-[r0 :T2]-(n1), (n2 :L6)<-[r1 :T2]-(n3 :L0) WHERE ...
2 UNWIND [-1465465557] AS a0
3 MATCH (n4 :L0)<-[r2 :T2]-(n5 :L0 :L6) WHERE ...
4 UNWIND [(n0.k65)] AS a1
5 RETURN (r1.k86) AS a2, (n3.k4) AS a3, (r1.k87) AS a4 ORDER BY a4 DESC
6 // expected result: {a2:0spkB,a3:false,a4:SqpUzADY6} ✓
7 // actual result: {} ✖
```

Figure: A logic bug in Memgraph caused by a bad optimization plan (Fig. 8 from paper).

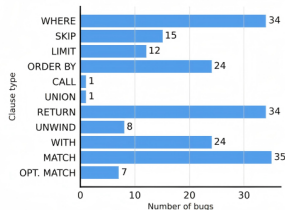
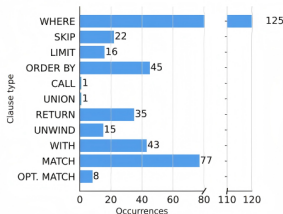
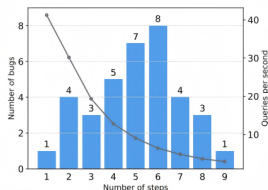
The Synthesized Query:

- A 5-step query with 8 clauses.
- Triggers a complex combination of query optimizations.

The Bug:

- Memgraph returns an **empty result set**.
- The expected result is a single row with three specific values.

Experiment: Bug Distribution by Query Features



Synthesis steps (Fig 10):

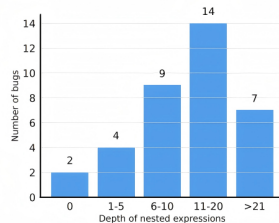
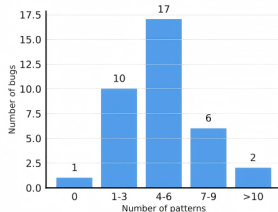
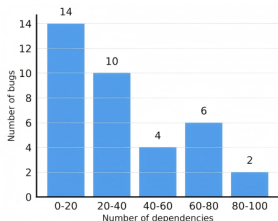
- 80% of bugs are triggered by queries with ≥ 3 steps.
- Queries with **4–6 steps** are most effective.
- Fewer-step queries still expose bugs in core functions (e.g., Example 5.3).

Clause types involved (Fig 11 & 12):

- MATCH appears in >70 queries; WHERE in >100 (used in both MATCH and WITH).
- 24 out of 36 bugs involve WITH or ORDER BY, not just the basic MATCH-WHERE-RETURN.

Conclusion: Complex, multi-step queries with diverse clauses are essential for effective bug detection.

Experiment: Impact of Query Complexity



Cross-step dependencies (Fig 13):

- Over **61%** of bugs come from queries with **>20 dependencies**.
- Such queries stress query planners during optimization.

Search patterns and expressions (Fig 14 & 15):

- **Two-thirds** of bugs are triggered by queries with **>3 distinct patterns**.
- **83%** of bugs involve expressions with **>5 levels of nesting**.

Conclusion: High pattern diversity and deeply nested expressions are crucial for exposing logic bugs.

GQS vs. State-of-the-Art Testers

GQS generates significantly more complex queries.

Tester	Patterns	Expr. Depth	Clauses	Dependencies
GDsmith	4.96	3.68	6.39	21.75
GDBMeter	0.86	2.24	1.94	1.97
Gamera	0.83	1.39	1.92	1.89
GQT	1.03	2.87	3.39	3.43
GRev	6.69	5.26	6.49	28.41
GQS	8.14	7.82	6.50	56.02

Table: Average complexity of test queries (Table 5 from paper).

24-Hour Test: GQS found 13 bugs (11 logic), while the best competitor found only 7 (6 logic).

Summary and Future Work

Summary:

- **GQS** is the first GDB testing approach based on a **ground truth oracle**.
- The stepwise synthesis generates highly complex queries that effectively uncover deep logic bugs.
- GQS found 36 new bugs, demonstrating superior effectiveness over existing methods.

Summary and Future Work

Summary:

- **GQS** is the first GDB testing approach based on a **ground truth oracle**.
- The stepwise synthesis generates highly complex queries that effectively uncover deep logic bugs.
- GQS found 36 new bugs, demonstrating superior effectiveness over existing methods.

Limitations and Future Work:

- Currently supports only the common subset of Cypher. Future: support GDB-specific features.
- No automated root cause analysis or bug deduplication.
- Extend beyond logic bugs: performance testing, concurrency/isolation checking.
- Adapt the methodology to other graph query languages (Gremlin, GQL).

Thank You!

Code: <https://github.com/Graph-Query-Synthesis/GQS>

Paper: <https://doi.org/10.1145/3749186>